

Jugar <https://visualgo.net/en>

Clase 1: Repaso de estructuras de datos y notación asintótica

Lo que traen C++ y Python; notaciones O , Ω y Θ

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Julio de 2026

Objetivos de la sesión I

Al terminar la clase, usted podrá

- Reconocer las estructuras de datos que ya traen C++ y Python y el costo de sus operaciones (OA1).
- Estimar el costo de un código contando operaciones línea a línea (OA1).
- Enunciar las definiciones de O , Ω y Θ (CLRS, Sección 3.1) (OA6).
- Escribir demostraciones de pertenencia a O con la estructura del curso: teorema, estrategia, desarrollo y conclusión con testigos (OA6).

Objetivos de la sesión II

Conocimiento previo

Se asume el curso de Estructuras de Datos: qué es una pila, una cola, una lista, un diccionario. Hoy no se reconstruyen esas estructuras; se repasa lo que cada lenguaje trae listo para usarlas.

Contenido I

- 1 Repaso de estructuras de datos
- 2 El costo de un algoritmo
- 3 Crecimiento de funciones y notación O
- 4 Cómo se escribe una demostración
- 5 Las notaciones Ω y Θ
- 6 Terminología y peor caso
- 7 Ejercicios
- 8 Cierre

Contenido I

- 1 Repaso de estructuras de datos
- 2 El costo de un algoritmo
- 3 Crecimiento de funciones y notación O
- 4 Cómo se escribe una demostración
- 5 Las notaciones Ω y Θ
- 6 Terminología y peor caso
- 7 Ejercicios
- 8 Cierre

¿Qué estructura conviene? I

Tres situaciones

- 1 Deshacer la última acción en un editor de texto (Ctrl+Z).
- 2 Atender los turnos de una fila en un banco.
- 3 Buscar el teléfono de un contacto a partir de su nombre.

Preguntas

¿Qué estructura usaría en cada caso? ¿Cuánto cuesta la operación principal de cada una?

¿Qué estructura conviene? Respuestas I

Tres situaciones, tres estructuras

- 1 Ctrl+Z deshace *lo último* que se hizo: pila (LIFO), push y pop en $O(1)$.
- 2 La fila atiende *al primero* que llegó: cola (FIFO), entrar y salir en $O(1)$.
- 3 El nombre es la *clave* y el teléfono el *valor*: diccionario, consulta en $O(1)$.

El plan del repaso

Todas estas estructuras ya vienen implementadas en C++ y en Python. El repaso recorre qué ofrece cada una y cuánto cuesta cada operación, recordando de paso el porqué de cada costo.

Arreglos



Lista



```
int arr[]  
list<int> list;
```

arr ←

arr + 0

arr[0]

arr[10⁶]

arr + 1 × 32

arr[1]

Listas I

Qué es

Secuencia de valores que pueden ser recorridos en el orden en el que están en la secuencia. La implementación usual es una estructura enlazada; `std::list` usa una lista doblemente enlazada circular.

Operaciones

`push_front`, `push_back`, `pop_front`, `pop_back`, `insert`, `erase`, `front`, `back`, `size`, `empty`.

Listas: operaciones y costos I

Operación	<code>std::list</code>	<code>std::deque</code>	<code>deque</code> (Python)	Costo
<code>push_back</code>	<code>l.push_back(x)</code>	<code>l.push_back(x)</code>	<code>l.append(x)</code>	$O(1)$
<code>push_front</code>	<code>l.push_front(x)</code>	<code>l.push_front(x)</code>	<code>l.appendleft(x)</code>	$O(1)$
<code>pop_back</code>	<code>l.pop_back()</code>	<code>l.pop_back()</code>	<code>l.pop()</code>	$O(1)$
<code>pop_front</code>	<code>l.pop_front()</code>	<code>l.pop_front()</code>	<code>l.popleft()</code>	$O(1)$
<code>front</code>	<code>l.front()</code>	<code>l.front()</code>	<code>l[0]</code>	$O(1)$
<code>back</code>	<code>l.back()</code>	<code>l.back()</code>	<code>l[-1]</code>	$O(1)$
<code>insert</code>	<code>l.insert(it, x)</code>	<code>l.insert(it, x)</code>	<code>l.insert(pos, x)</code>	$O(k)$
<code>erase</code>	<code>l.erase(it)</code>	<code>l.erase(it)</code>	<code>del l[pos]</code>	$O(k)$
<code>size</code>	<code>l.size()</code>	<code>l.size()</code>	<code>len(l)</code>	$O(1)$
<code>empty</code>	<code>l.empty()</code>	<code>l.empty()</code>	<code>len(l) == 0</code>	$O(1)$

Listas: operaciones y costos II

El porqué, en una línea

Hay referencias directas a ambos extremos: por eso todo lo que toca solo los extremos es $O(1)$. Insertar o borrar en la posición k exige primero llegar hasta ella recorriendo la secuencia: $O(k)$.

Listas en código I

```
from collections import deque

l = deque()
l.append(10)           # push_back,  O(1)
l.appendleft(5)        # push_front, O(1)
ultimo = l.pop()       # pop_back,   O(1)
primero = l.popleft()  # pop_front,  O(1)
```

```
#include <deque>
using namespace std;
```

```
deque<int> l;
l.push_back(10); // O(1)
l.push_front(5); // O(1)
l.pop_back();    // O(1)
l.pop_front();   // O(1)
```

Listas en código II

Para ejecutar en clase

Los dos programas completos están en `codigo/listas.py` y `codigo/listas.cpp`; el segundo se compila con `g++ -std=c++17 -o listas listas.cpp`.

Pilas I

Qué es

Secuencia de valores en la que solo se tiene acceso por un extremo, el tope. Política LIFO: *last in, first out*. Referencia: CLRS, Sección 10.1, p. 232.

Operación	<code>std::vector</code>	<code>std::stack</code>	<code>deque</code> (Python)	Costo
push	<code>t.push_back(x)</code>	<code>t.push(x)</code>	<code>t.append(x)</code>	$O(1)^*$
top	<code>t[t.size()-1]</code>	<code>t.top()</code>	<code>t[-1]</code>	$O(1)$
pop	<code>t.pop_back()</code>	<code>t.pop()</code>	<code>t.pop()</code>	$O(1)$
size	<code>t.size()</code>	<code>t.size()</code>	<code>len(t)</code>	$O(1)$
empty	<code>t.empty()</code>	<code>t.empty()</code>	<code>len(t) == 0</code>	$O(1)$

Pilas II

El porqué, en una línea

Toda operación toca únicamente el tope. El asterisco: en vector, `push_back` es $O(1)$ *amortizado*, porque de vez en cuando el arreglo interno se redimensiona.

Qué es

Secuencia con acceso por los dos extremos: por uno se agregan elementos y por el otro se remueven. Política FIFO: *first in, first out*. Referencia: CLRS, Sección 10.1, p. 232.

Operación	<code>std::queue</code>	<code>deque</code> (Python)	Costo
push	<code>t.push(x)</code>	<code>t.append(x)</code>	$O(1)$
front	<code>t.front()</code>	<code>t[0]</code>	$O(1)$
pop	<code>t.pop()</code>	<code>t.popleft()</code>	$O(1)$
size	<code>t.size()</code>	<code>len(t)</code>	$O(1)$
empty	<code>t.empty()</code>	<code>len(t) == 0</code>	$O(1)$

Error común en Python

Usar la lista nativa como cola con `l.pop(0)`: al sacar el primero, *todos* los demás se desplazan una posición y la operación cuesta $O(n)$. Para colas, `collections.deque`.

Tablas de direccionamiento directo I

Qué es

Estructura que asocia claves a valores; la clave determina la ubicación en la que se almacena el valor. El requisito: obtener el valor a partir de su clave de forma eficiente. Referencia: CLRS, Sección 11.1, p. 254.

Implementaciones

Los arreglos y vectores de C++ (claves = índices), los mapas de C++, las listas nativas de Python (claves = índices) y los diccionarios de Python.

Operación	vector (C++)	map (C++)	list (Python)	dict (Python)
assign	<code>t.push_back(v)</code> $O(1)^*$	<code>t[c] = v</code> $O(\log n)$	<code>t.append(v)</code> $O(1)^*$	<code>t[c] = v</code> $O(1)$
update	<code>t[i] = v</code> $O(1)$	<code>t[c] = v</code> $O(\log n)$	<code>t[i] = v</code> $O(1)$	<code>t[c] = v</code> $O(1)$
query	<code>t[i]</code> $O(1)$	<code>t[c]</code> $O(\log n)$	<code>t[i]</code> $O(1)$	<code>c in t</code> $O(1)$

$$\alpha = 3$$



$$h(k) = k \bmod 10$$

$$11 \bmod 10 = 1$$

$$23 \bmod 10 = 3$$

$$101 \bmod 10 = 1$$

8083

$$2) h(8083) = 3$$

$$O(1 + \alpha)$$

Tablas de direccionamiento directo II

El porqué, en una línea

Con índices, la posición se calcula con un desplazamiento directo en memoria: $O(1)$. El asterisco de `push_back` y `append`: $O(1)$ amortizado, por el redimensionamiento ocasional del arreglo. El `map` de C++ guarda un árbol rojo-negro balanceado: $O(\log n)$ (CLRS, cap. 13). El `dict` de Python usa una tabla hash: $O(1)$ en promedio (CLRS, Sección 11.2).

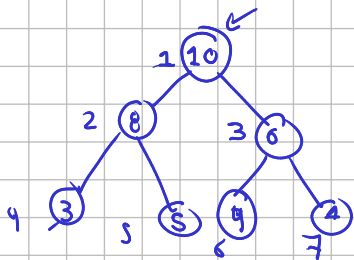
Colas de prioridad I

Qué es

Cola en la que los elementos no salen en el orden en que entraron, sino de acuerdo con una prioridad. La prioridad puede ser el valor mismo del elemento o una cantidad calculada sobre él.

Implementación usual: montículos (*heaps*) u otras estructuras con operaciones $O(\log n)$. Referencia: CLRS, Sección 6.5, p. 162.

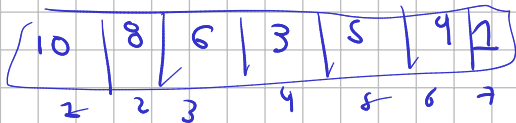
Operación	priority_queue (C++)	heapq (Python)	Costo
push	t.push(x)	heappush(t, x)	$O(\log n)$
top	t.top()	t[0]	$O(1)$
pop	t.pop()	heappop(t)	$O(\log n)$
size	t.size()	len(t)	$O(1)$
empty	t.empty()	len(t) == 0	$O(1)$



$$h[2i] = 2i$$

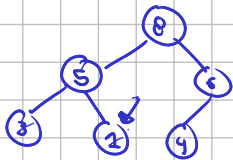
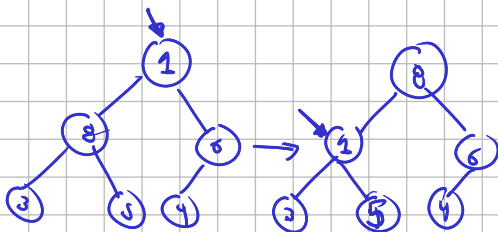
$$h[2i+1] = 2i+1$$

$$p = \lfloor \frac{i}{2} \rfloor$$



pop

1 8 6 3 5 4



Colas de prioridad II

Cuidado al traducir entre lenguajes

`priority_queue` de C++ entrega primero el *mayor*, `heapq` de Python entrega primero el *menor*. Para invertir el orden: negar los valores o cambiar el comparador.

Conjuntos I

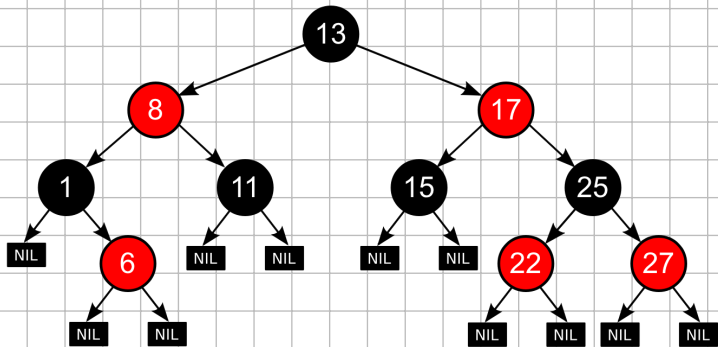
Qué es

Agrupación de elementos sin repetidos y sin orden entre ellos, como los conjuntos matemáticos.

Operación	set (C++)	set (Python)	Costo C++	Costo Python
insert	<code>t.insert(x)</code>	<code>t.add(x)</code>	$O(\log n)$	$O(1)$
erase	<code>t.erase(x)</code>	<code>t.discard(x)</code>	$O(\log n)$	$O(1)$
size	<code>t.size()</code>	<code>len(t)</code>	$O(1)$	$O(1)$
empty	<code>t.empty()</code>	<code>len(t) == 0</code>	$O(1)$	$O(1)$

El porqué, en una línea

Igual que en los mapas: set de C++ es un árbol rojo-negro ($O(\log n)$); set de Python es una tabla hash ($O(1)$ en promedio).



Resumen del repaso I

Estructura	C++	Python	Operaciones dominantes
Lista	<code>list</code> , <code>deque</code>	<code>collections.deque</code>	extremos $O(1)$; posición k : $O(k)$
Pila	<code>stack</code> , <code>vector</code>	<code>deque</code> , <code>list</code>	<code>push/pop</code> $O(1)$
Cola	<code>queue</code>	<code>deque</code>	<code>push/pop</code> $O(1)$
Direcc. directo	<code>vector</code> , <code>map</code>	<code>list</code> , <code>dict</code>	índice $O(1)$; <code>map</code> $O(\log n)$; <code>dict</code> $O(1)$
Cola de prioridad	<code>priority_queue</code>	<code>heapq</code>	<code>push/pop</code> $O(\log n)$; <code>top</code> $O(1)$
Conjunto	<code>set</code>	<code>set</code>	<code>insert/erase</code> $O(\log n)$ / $O(1)$

Nota sobre los costos con hash

Los costos $O(1)$ de `dict` y `set` de Python son en promedio; el peor caso es mayor, pero en la práctica del curso se toman como constantes.

Ejercicio: elija la estructura I

Para cada situación, elija estructura y justifique con el costo

- 1 Verificar si un usuario ya fue procesado antes.
- 2 Atender solicitudes según su urgencia, no según su llegada.
- 3 Recordar las últimas k posiciones visitadas, descartando las viejas.

Ejercicio: elija la estructura. Respuestas I

Respuestas

- 1 Conjunto: preguntar si el usuario está cuesta $O(1)$ en Python ($O(\log n)$ con set de C++).
- 2 Cola de prioridad: sale primero el más urgente, con push y pop en $O(\log n)$.
- 3 deque: se agrega por un extremo en $O(1)$ y, cuando hay más de k , se descarta por el otro en $O(1)$.

Contenido I

- 1 Repaso de estructuras de datos
- 2 El costo de un algoritmo
- 3 Crecimiento de funciones y notación O
- 4 Cómo se escribe una demostración
- 5 Las notaciones Ω y Θ
- 6 Terminología y peor caso
- 7 Ejercicios
- 8 Cierre

¿De dónde salen esos costos? I

La pregunta de la segunda parte

Las tablas del repaso están llenas de símbolos $O(1)$, $O(\log n)$, $O(k)$. ¿Qué significa exactamente $O(\cdot)$? ¿Cómo se demuestra que un costo pertenece a $O(\cdot)$?

Primera intuición

Antes de la definición formal: el costo de un algoritmo se estima *contando operaciones*. Referencia: CLRS, Sección 2.2, p. 23.

Contar operaciones línea a línea I

```
def suma(datos):  
    total = 0  
    i = 0  
    while i < len(datos):  
        total = total + datos[i]  
        i = i + 1  
    return total
```

Línea	Costo	Veces
total = 0	c_1	1
i = 0	c_2	1
while i < len(datos)	c_3	$n + 1$
total = total + datos[i]	c_4	n
i = i + 1	c_5	n
return total	c_6	1

$3n + 4$

Contar operaciones línea a línea II

Total

$T(n) = (c_3 + c_4 + c_5) n + (c_1 + c_2 + c_3 + c_6) = a n + b$: una recta en n .

Para verificar el conteo

`codigo/conteo_operaciones.py` ejecuta la misma función con un contador por línea y reproduce esta tabla: al duplicar n , el total de operaciones se duplica.

¿Importan a y b ? I

Pregunta

Si se ejecuta suma en un computador el doble de rápido, a y b se reducen a la mitad. ¿Cambia la forma en que crece $T(n)$?

¿Importan a y b ? Respuesta I

Respuesta

No: $T(n)$ sigue siendo una recta, y para n grande el término $a n$ domina sobre b . Lo que distingue a los algoritmos es la *forma de crecimiento* de $T(n)$, no las constantes. Para comparar formas de crecimiento ignorando constantes se usa la notación asintótica (CLRS, Sección 3.1).

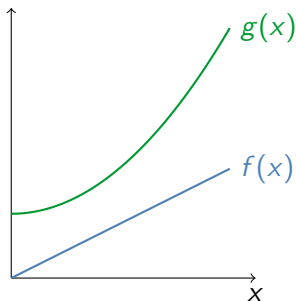
Contenido I

- 1 Repaso de estructuras de datos
- 2 El costo de un algoritmo
- 3 Crecimiento de funciones y notación O**
- 4 Cómo se escribe una demostración
- 5 Las notaciones Ω y Θ
- 6 Terminología y peor caso
- 7 Ejercicios
- 8 Cierre

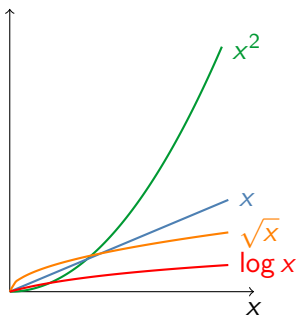
Cota superior I

Idea

$g(x)$ es una **cota superior** de $f(x)$ si siempre se cumple que $f(x) \leq g(x)$, para cualquier valor de x .



Varias funciones bajo $g(x) = x^2$



Lectura

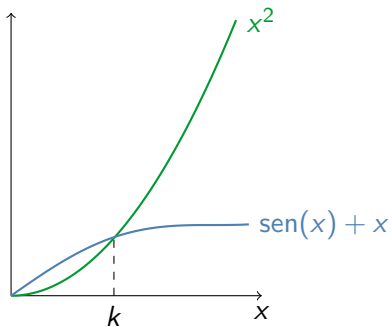
Desde cierto punto en adelante, x^2 queda por encima de x , de $\sqrt{x}$ y de $\log x$: una misma función puede acotar a muchas a la vez. Cerca del origen las curvas se cruzan; enseguida se ve qué hacer con ese arranque.

¿Y si la función oscila? I

Pregunta

¿Es $g(x) = x^2$ una cota superior de $f(x) = \sin(x) + x$?

¿Y si la función oscila? Respuesta I



¿Y si la función oscila? Respuesta II

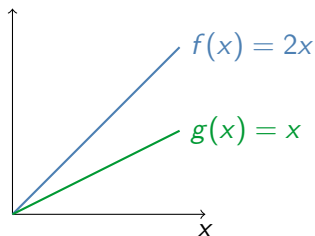
Respuesta

Cerca del origen la cota falla: en $x = 1$, $\sin(1) + 1 \approx 1,84 > 1 = x^2$. Pero desde cierto valor k en adelante, x^2 queda siempre por encima. En el análisis asintótico se usa esa noción más flexible: la desigualdad debe valer **para todos los** $x \geq k$, y pueden existir valores $x < k$ donde no valga. Es una cota *asintótica*, no global.

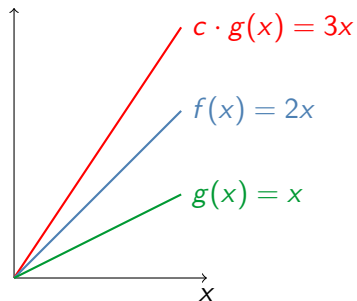
¿Y si la cota no alcanza? I

Pregunta

¿Es $g(x) = x$ una cota superior de $f(x) = 2x$? Si no lo es, ¿se puede reparar?



¿Y si la cota no alcanza? Respuesta I



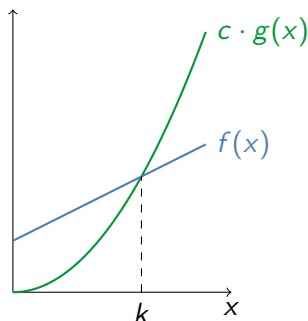
Respuesta

No: $2x > x$ para todo $x > 0$. Pero al multiplicar g por una constante, $c \cdot g(x) = 3x$ sí es cota superior de $2x$. La constante c absorbe los factores que no cambian la forma de crecimiento.

Las dos ideas juntas I

Cota superior asintótica

Si $g(x)$, o algún múltiplo $c \cdot g(x)$, es una cota superior de $f(x)$ para todos los $x \geq k$, se dice que $f(x)$ es $O(g(x))$. c y k son constantes.



Definición formal de O I

Definición (CLRS, Sección 3.1, p. 47)

$O(g(n)) = \{f(n) : \text{existen constantes positivas } c \text{ y } k \text{ tales que}$
 $0 \leq f(n) \leq c \cdot g(n) \text{ para todo } n \geq k\}$

Definición formal de O II

Observaciones

- $O(g(n))$ es un *conjunto de funciones*; por eso se escribe $f(n) \in O(g(n))$. CLRS escribe $f(n) = O(g(n))$, un abuso de notación aceptado.
- CLRS llama n_0 a la constante que aquí llamamos k . En el curso usamos k .
- A las constantes c y k las llamamos **testigos**: exhibirlas es lo que demuestra la pertenencia.
- La notación viene de los trabajos de Landau y fue popularizada en computación por Donald Knuth.

Ejemplo inmediato

$O(x^2)$ contiene, entre otras, a x , $\sqrt{x}$, $\log x$, $\sin(x) + x$ y $50x + 7$.

Contenido I

- 1 Repaso de estructuras de datos
- 2 El costo de un algoritmo
- 3 Crecimiento de funciones y notación O
- 4 Cómo se escribe una demostración**
- 5 Las notaciones Ω y Θ
- 6 Terminología y peor caso
- 7 Ejercicios
- 8 Cierre

La estructura de una demostración en el curso I

Cuatro partes, siempre

- 1 **Teorema:** enunciado preciso de lo que se quiere demostrar.
- 2 **Demostración:** se declara la estrategia antes de desarrollar (“se procede de forma directa”, “se procede por contradicción”).
- 3 **Desarrollo:** la cadena de desigualdades; aquí aparecen los testigos c y k .
- 4 **Conclusión:** cierre explícito: “por lo tanto, se puede concluir que ... con testigos $c = \dots$ y $k = \dots$ ”.

Con esta misma estructura se escriben las demostraciones de los quices, los talleres y los parciales.

Buscando testigos por tanteo I

Queremos demostrar

$3n^2 \in O(n^2)$: hay que encontrar c y k con $3n^2 \leq c \cdot n^2$ para todo $n \geq k$.

c	$\text{¿}3n^2 \leq c \cdot n^2 \text{ para todo } n \geq 1\text{?}$	Veredicto
1	$3n^2 \leq n^2$ es falso para $n \geq 1$	×
2	$3n^2 \leq 2n^2$ es falso para $n \geq 1$	×
3	$3n^2 \leq 3n^2$ vale para todo $n \geq 1$	✓

Testigos encontrados

$c = 3$ y $k = 1$. Ahora se escribe la demostración completa.

La demostración escrita I

Teorema

$$3n^2 \in O(n^2).$$

Demostración

Se procede de forma directa.

Desarrollo. Sea $n \geq 1$. Entonces

$$0 \leq 3n^2 \leq 3 \cdot n^2,$$

de modo que la definición de $O(n^2)$ se cumple con $c = 3$ y $k = 1$.

Conclusión. Por lo tanto, se puede concluir que $3n^2 \in O(n^2)$ con testigos $c = 3$ y $k = 1$. ■

Mismo molde, otros datos I

Teorema

$$7x^2 \in O(x^3).$$

Demostración

Se procede de forma directa.

Desarrollo. Sea $x \geq 7$. Entonces $7 \leq x$ y, multiplicando ambos lados por $x^2 > 0$,

$$7x^2 \leq x \cdot x^2 = 1 \cdot x^3,$$

de modo que la definición se cumple con $c = 1$ y $k = 7$.

Conclusión. Por lo tanto, se puede concluir que $7x^2 \in O(x^3)$ con testigos $c = 1$ y $k = 7$. ■

¿Y al revés? I

Pregunta

¿Es $x^3 \in O(7x^2)$? Intente encontrar testigos antes de continuar.

Cuando no hay testigos: contradicción I

Teorema

$$x^3 \notin O(7x^2).$$

Demostración

Se procede por contradicción.

Desarrollo. Supongamos que $x^3 \in O(7x^2)$: existen constantes positivas c y k tales que $x^3 \leq c \cdot 7x^2$ para todo $x \geq k$. Dividiendo por $x^2 > 0$:

$$x \leq 7c \quad \text{para todo } x \geq k.$$

Pero x crece sin límite: en $x = \max(k, 7c) + 1$ la desigualdad falla. Contradicción.

Conclusión. Por lo tanto, se puede concluir que $x^3 \notin O(7x^2)$. ■

Errores comunes al demostrar I

Tres errores que cuestan puntos

- Evaluar en un solo valor (“en $x = 10$ sí se cumple”) no demuestra nada: la desigualdad debe valer para *todos* los $x \geq k$.
- Los testigos son constantes: un “testigo” que depende de n (como $c = n$) no es un testigo.
- En las demostraciones de pertenencia, una conclusión sin testigos está incompleta: el cierre dice con qué c y qué k se concluye. En una no pertenencia no hay testigos que exhibir.

Contenido I

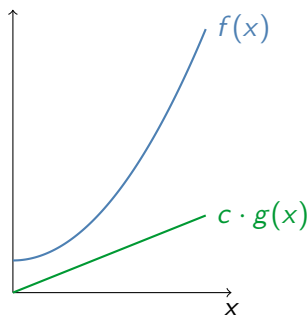
- 1 Repaso de estructuras de datos
- 2 El costo de un algoritmo
- 3 Crecimiento de funciones y notación O
- 4 Cómo se escribe una demostración
- 5 Las notaciones Ω y Θ**
- 6 Terminología y peor caso
- 7 Ejercicios
- 8 Cierre

Cota inferior: notación Ω

Definición (CLRS, Sección 3.1, p. 48)

$$\Omega(g(n)) = \{f(n) : \text{existen constantes positivas } c \text{ y } k \text{ tales que} \\ 0 \leq c \cdot g(n) \leq f(n) \text{ para todo } n \geq k\}$$

Cota inferior: notación Ω



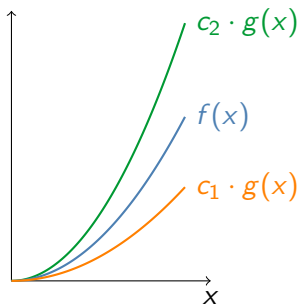
Lectura

Si el tiempo de un algoritmo está en $\Omega(n)$, sus tiempos de ejecución siempre quedan por encima de la recta $c \cdot n$ desde k en adelante: g acota *por debajo*.

Cota ajustada: notación Θ I

Teorema 3.1 (CLRS, p. 48)

Para dos funciones $f(n)$ y $g(n)$ cualesquiera, $f(n) \in \Theta(g(n))$ si y solo si $f(n) \in O(g(n))$ y $f(n) \in \Omega(g(n))$.



Cota ajustada: notación Θ II

Ejemplo inmediato

$3n^2 \in \Theta(n^2)$: la pertenencia a $O(n^2)$ ya se demostró con testigos $c = 3$ y $k = 1$; la pertenencia a $\Omega(n^2)$ sale de $1 \cdot n^2 \leq 3n^2$, con testigos $c = 1$ y $k = 1$.

En este curso

Las tres notaciones quedan presentadas; a lo largo del curso se trabaja principalmente con O .

Contenido I

- 1 Repaso de estructuras de datos
- 2 El costo de un algoritmo
- 3 Crecimiento de funciones y notación O
- 4 Cómo se escribe una demostración
- 5 Las notaciones Ω y Θ
- 6 Terminología y peor caso**
- 7 Ejercicios
- 8 Cierre

Terminología de complejidades I

Complejidad	Terminología
$O(1)$	constante
$O(\log n)$	logarítmica
$O(n)$	lineal
$O(n \log n)$	$n \log n$
$O(n^b)$	polinomial
$O(b^n)$	exponencial
$O(n!)$	factorial

De vuelta a las tablas del repaso

top de una cola de prioridad: $O(1)$, constante. push: $O(\log n)$, logarítmica. insert en una lista en la posición k : $O(k)$, lineal en la posición.

La cota se toma sobre el peor caso I

Convención

Cuando se reporta que el tiempo de un algoritmo es $O(g(n))$, el análisis se hace sobre el **peor caso**: así la cota no puede resultar peor de lo anunciado.

Pregunta

Para el algoritmo 1 se encontró $T_1(n) \in O(n^2)$ y para el algoritmo 2, $T_2(n) \in O(n \log n)$. ¿Se puede asegurar que el algoritmo 2 siempre es más rápido que el 1?

MergeSort $O(n \log(n))$

InsertionSort $O(n^2)$

Mejor caso $O(n)$

La cota se toma sobre el peor caso. Respuesta I

Respuesta

No. Las cotas hablan del peor caso y esconden las constantes: para entradas pequeñas, un algoritmo $O(n^2)$ con constantes chicas puede ganarle a uno $O(n \log n)$ con constantes grandes. La garantía es asintótica: desde algún k en adelante.

Abuso de lenguaje frecuente

El tiempo del ordenamiento por inserción no es una única función de n : cambia con la entrada. La cota $O(n^2)$ sí vale para toda entrada, porque acota por encima. El abuso está en decir que el tiempo es $\Theta(n^2)$ a secas: con la entrada ya ordenada el tiempo es lineal. Lo preciso: el **peor caso** es $\Theta(n^2)$. El análisis de mejor caso, caso medio y análisis amortizado se estudia en el curso siguiente (Análisis de Algoritmos).

Contenido I

- 1 Repaso de estructuras de datos
- 2 El costo de un algoritmo
- 3 Crecimiento de funciones y notación O
- 4 Cómo se escribe una demostración
- 5 Las notaciones Ω y Θ
- 6 Terminología y peor caso
- 7 Ejercicios**
- 8 Cierre

Ejercicios: demuestre con la estructura del curso I

Batería

Demuestre, escribiendo teorema, estrategia, desarrollo y conclusión con testigos:

- 1 $x^2 + 2x + 1 \in O(x^2)$.
- 2 $\log_2 n \in O(n)$; parta de que $n < 2^n$.
- 3 $x^2 + 4x + 17 \in O(x^3)$, y además $x^3 \notin O(x^2 + 4x + 17)$.
- 4 $x \log_2 x \in O(x^2)$.
- 5 Explique qué significa que una función sea $O(1)$ y qué significa que sea $\Theta(1)$.

Pistas y soluciones generales I

Para verificar el rumbo

- 1 Acote cada término por el dominante:
 $x^2 + 2x + 1 \leq x^2 + 2x^2 + x^2 = 4x^2$ para $x \geq 1$. Testigos $c = 4$, $k = 1$.
- 2 Tome logaritmo en base 2 a ambos lados de $n < 2^n$:
 $\log_2 n < n$. Testigos $c = 1$, $k = 1$.
- 3 Misma técnica del punto 1: $x^2 + 4x + 17 \leq 22x^3$ para $x \geq 1$ (testigos $c = 22$, $k = 1$). Para la no pertenencia, imite la demostración por contradicción de $x^3 \notin O(7x^2)$.

Pistas y soluciones generales II

Para verificar el rumbo (continuación)

- 4 Use $\log_2 x \leq x$ para $x \geq 1$ y multiplique por x . Testigos $c = 1$, $k = 1$.
- 5 Revise la definición con $g(n) = 1$: el valor de la función queda acotado por la constante c desde k en adelante; no crece con la entrada. Con $\Theta(1)$ queda además acotado por debajo.

Ejercicio de cierre I

Para resolver por su cuenta

Demuestre que $2^{n+1} \in O(2^n)$ y decida, con demostración, si $2^{2n} \in O(2^n)$.

Contenido I

- 1 Repaso de estructuras de datos
- 2 El costo de un algoritmo
- 3 Crecimiento de funciones y notación O
- 4 Cómo se escribe una demostración
- 5 Las notaciones Ω y Θ
- 6 Terminología y peor caso
- 7 Ejercicios
- 8 Cierre**

Ideas de la clase

- Las estructuras del curso ya vienen en C++ y Python; lo que se decide es cuál usar, mirando el costo de sus operaciones.
- deque da extremos en $O(1)$; dict y set de Python consultan en $O(1)$; map, set y la cola de prioridad trabajan en $O(\log n)$.
- $f(n) \in O(g(n))$ si existen testigos c y k con $f(n) \leq c \cdot g(n)$ para todo $n \geq k$.
- Toda demostración: teorema, estrategia, desarrollo y conclusión; al demostrar pertenencia, la conclusión exhibe los testigos.
- Ω acota por debajo y Θ por ambos lados; el curso trabaja principalmente con O , siempre sobre el peor caso.

Referencias I



T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein.
Introduction to Algorithms, 3.^a ed., MIT Press, 2009.
Secciones 2.2 (análisis de algoritmos), 3.1 (notación asintótica), 6.5 (colas de prioridad), 10.1–10.2 (pilas, colas y listas enlazadas), 11.1–11.2 (direccionamiento directo y tablas hash) y cap. 13 (árboles rojo-negro).



Curso de Estructuras de Datos del profesor Carlos Ramírez:
<https://www.carlosramirez.info/teaching/estructuras-de-datos>



Documentación de referencia: cppreference.com y docs.python.org.